

VoiceIntent — Voice-to-Intent Pipeline for Customer Support

AI 620: Fundamentals of Data Engineering · LUMS · Final Project Report

Team

Member	Roll No.	Contribution
Hasan	{ROLL-1}	Data ingestion (HuggingFace + gTTS), PostgreSQL schema design, audio-metadata storage layer.
Rohan	{ROLL-2}	Whisper transcription pipeline, transcript cleaning, Great Expectations data-quality suite.
Lina	{ROLL-3}	ML pipeline: TF-IDF feature engineering, LogisticRegression intent classifier, evaluation + drift detection, threshold sweep.
Ibrahim Noor	{ROLL-4}	Prefect orchestration, FastAPI + Streamlit serving layer, gTTS response synthesis, Docker deployment, structured logging.

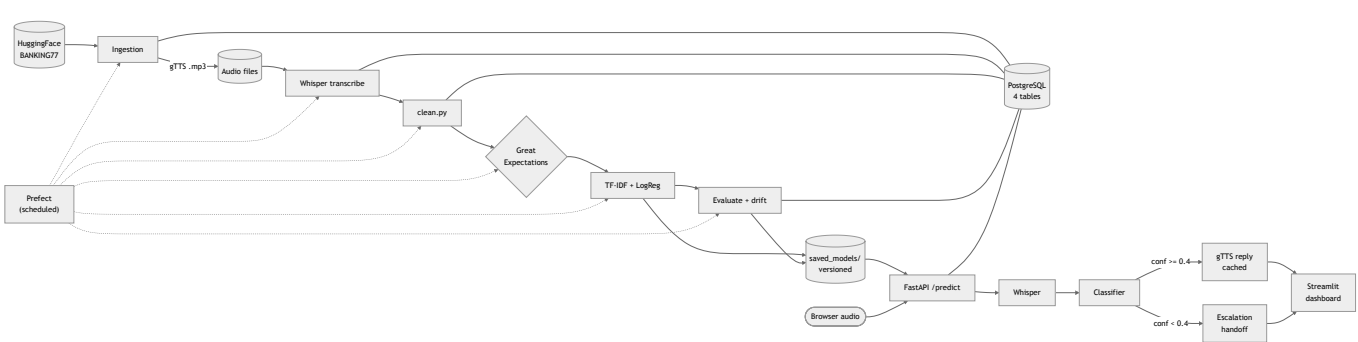
Project Description

Goal. Build a production-grade voice-to-intent pipeline that converts spoken banking customer queries into classified intents and synthesised voice replies. Domain: customer-support automation. Use case: an IVR-style agent that auto-handles confident calls and hands off uncertain ones to humans.

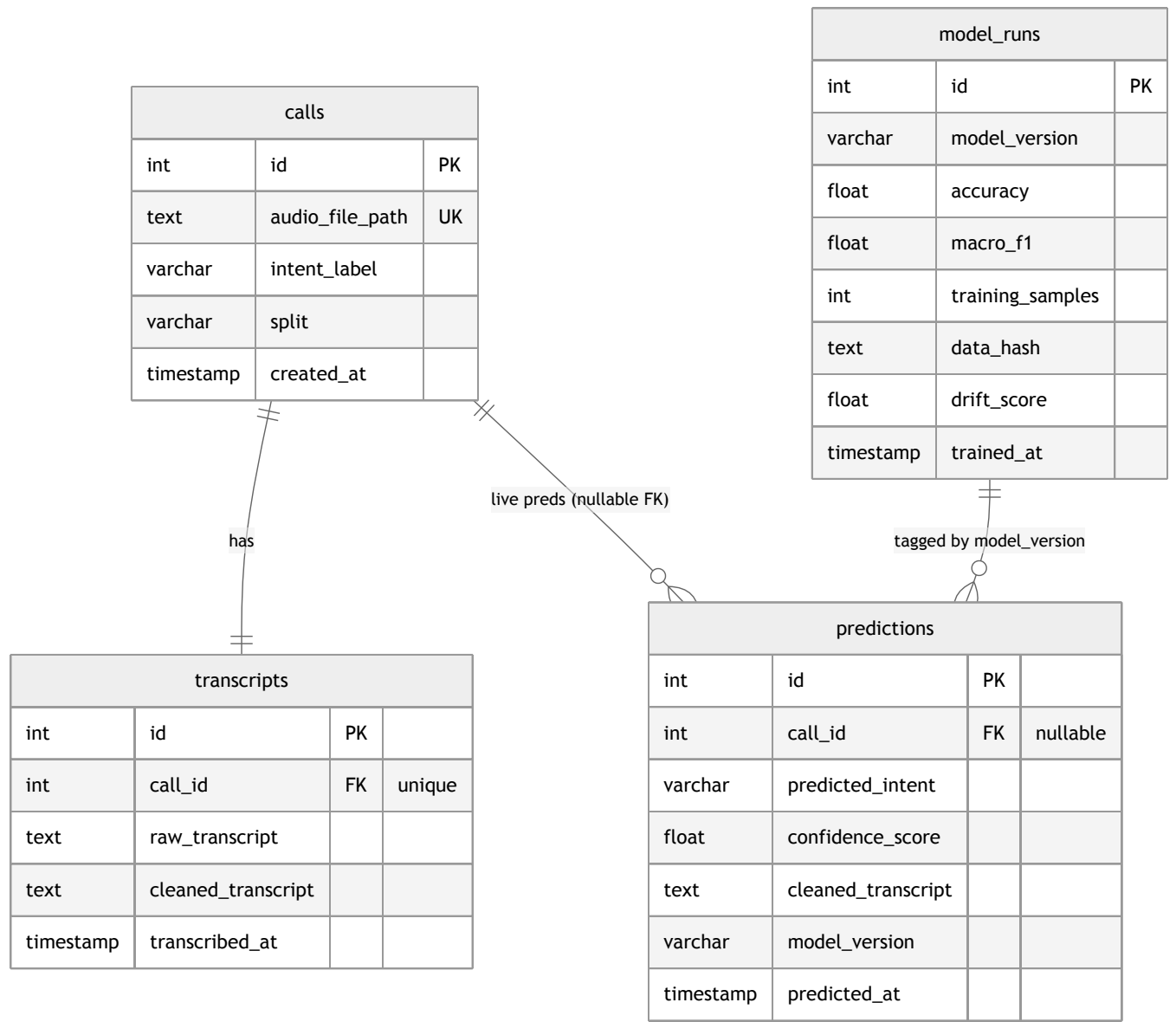
Dataset. BANKING77 from HuggingFace (`mteb/banking77`): 13,069 real customer queries across 77 fine-grained banking intents (e.g. `card_blocked`, `transfer_not_received_by_recipient`, `exchange_rate`). Text-only, so we synthesise corresponding audio with gTTS, then feed it back through Whisper to recover transcripts — exercising the full speech-to-text leg of a real call-centre stack.

TEST ACCURACY 87.91%	MACRO F1 0.8792	TRAINING-VS-TEST DRIFT 0.0907	AUTO-HANDLED AT $\Theta=0.4$ 73.5%
--------------------------------	---------------------------	---	--

Architecture



Database Schema



Why PostgreSQL. Relational fit (clear FK relationships, joins for evaluation queries), GX integrates natively via dataframe extraction, SQLAlchemy ORM gives us migration safety, and Postgres handles 13k rows trivially while remaining production-realistic.

Pipeline Stages & Tool Justification

Stage	Tool	Why
Ingestion	HuggingFace datasets + gTTS	Real public dataset; gTTS is rate-limit aware and lets us synthesise reproducible audio.
Storage	PostgreSQL 16 + SQLAlchemy	Strong typing, FK-enforced relationships, idempotent inserts via ON CONFLICT DO NOTHING .
Transcription	OpenAI Whisper (tiny/base)	Open-source, runs on CPU, good accuracy/latency trade-off; ffmpeg-backed audio decoding.
Cleaning	Custom clean_text	Lower-case, strip punctuation, mark [INAUDIBLE] ; idempotent (filters by cleaned_transcript IS NULL).
Quality	Great Expectations 1.x	5 expectations: not-null, length 5–500, intent in canonical 77-set, FK present, row count > 0. Run before training.
ML	scikit-learn TF-IDF + LogisticRegression	Strong baseline for 77-class text classification (88% accuracy), fast inference, calibrated probabilities for confidence-gated routing.
Orchestration	Prefect 3	Tasks have retries + delays; cron / interval scheduling via flow.serve(cron=...) ; UI & logs for run history.

Serving	FastAPI + Streamlit	Swagger-documented endpoints, in-browser mic recorder, live drift card, recent-calls panel.
Deployment	Docker Compose (CPU torch wheel)	One-command run; postgres + pipeline + api + dashboard each containerised; auto-loads dump on first start.
Logging	python-json-logger	Structured JSON to stdout + <code>logs/pipeline.log</code> ; HTTP middleware records latency per request.

Production-grade Extension: Model Deployment with Drift & Threshold Policy

Of the five advanced extensions, we implemented **#4 Production-grade Model Deployment**. Concretely:

- **Versioning:** models saved as `model_v{YYYYMMDD_HHMMSS}.pkl`; `model_runs` table tracks accuracy, F1, drift, and SHA-256 data hash per run; `latest_model.pkl` + `latest_model.txt` form a stable serving alias.
- **Drift detection (two layers):** training-vs-test JS divergence at evaluate time, *and* live rolling-window divergence over the most recent 100 vs prior 100 predictions via `GET /drift`, with top-mover intents surfaced.
- **Confidence-gated routing:** `CONFIDENCE_THRESHOLD=0.4`, justified by a sweep over `[0.10–0.90]` on the test set (`docs/threshold_sweep.png`): at 0.4 we escalate 23.3% of calls, the auto-handled subset is 95.9% precise, and only 3.2% of all calls receive a confidently-wrong canned reply. Below-threshold predictions get a synthesised "redirecting to an agent" voice handoff.
- **Operational hygiene:** idempotent schema migrations on API startup, disk-cached gTTS audio (95 entries, hot reload across restarts), pre-warmer thread fills the cache in the background, dashboard surfaces recent calls + drift + model version.

Repository, Deployment & AI Usage

Repository: <https://github.com/HasanAmjad/ai-voice-assistant>

Deployment: Local one-command via `docker-compose up --build` (postgres autoloads `voiceintent_dump.sql`, api+dashboard expose ports 8000 / 8501). No public deployment URL; full instructions in `README.md`.

AI Usage Disclosure.

- **Tool:** Claude (Anthropic).
- **Used for:** Debugging environment issues.
- **Extent:** Potential buggy code was given to Claude to debug and find issues.